TESTING SUMMARY REPORT

MIVA Level 300 Result Calculator System

1. INTRODUCTION

This report summarizes the unit testing process carried out on the **MIVA Level 300 Result Calculator System** using the **PyTest framework**.

The goal of the testing process was to ensure that all functions in the program work correctly, handle edge cases properly, and achieve a high level of test coverage using both **black-box and white-box testing techniques**.

2. TESTING APPROACH

The testing process was divided into two main categories:

2.1 Black-Box Testing

Black-box testing focuses on testing the functionality of the system without considering internal code structure.

The following techniques were used:

a. Equivalence Partitioning

Inputs were divided into valid and invalid groups:

- Valid inputs: `[70, 80, 90]`
- Mixed inputs: `[70, "80", 90]`
- Invalid inputs: `[10, "abc", 20]`

b. Boundary Value Analysis

Boundary values were tested to ensure correctness at limits:

- Minimum values: `0`
- Grade boundaries: `45, 50, 60, 70`
- Maximum values: `100`

2.2 White-Box Testing

White-box testing focuses on internal code structure and logic.

The following coverage techniques were implemented:

a. Statement Coverage

- Every line of code in `project.py` was executed during testing

b. Branch Coverage

- All decision paths in the `get_grade()` function were tested:
  - A, B, C, D, F conditions

c. Path Coverage

- Multiple execution paths were tested using different input combinations

3. TEST IMPLEMENTATION

Unit tests were written using the **PyTest framework** in a separate file:

- `test_project.py`

The following functions were tested:

- `calculate_average()`

- `get_grade()`

Test cases included:

- Normal inputs
- Boundary values
- Invalid inputs
- Edge cases (e.g., empty list)

4. TEST COVERAGE RESULTS

Test coverage was measured using the `pytest-cov` tool.

Coverage Command Used:

```
pytest --cov=project
```

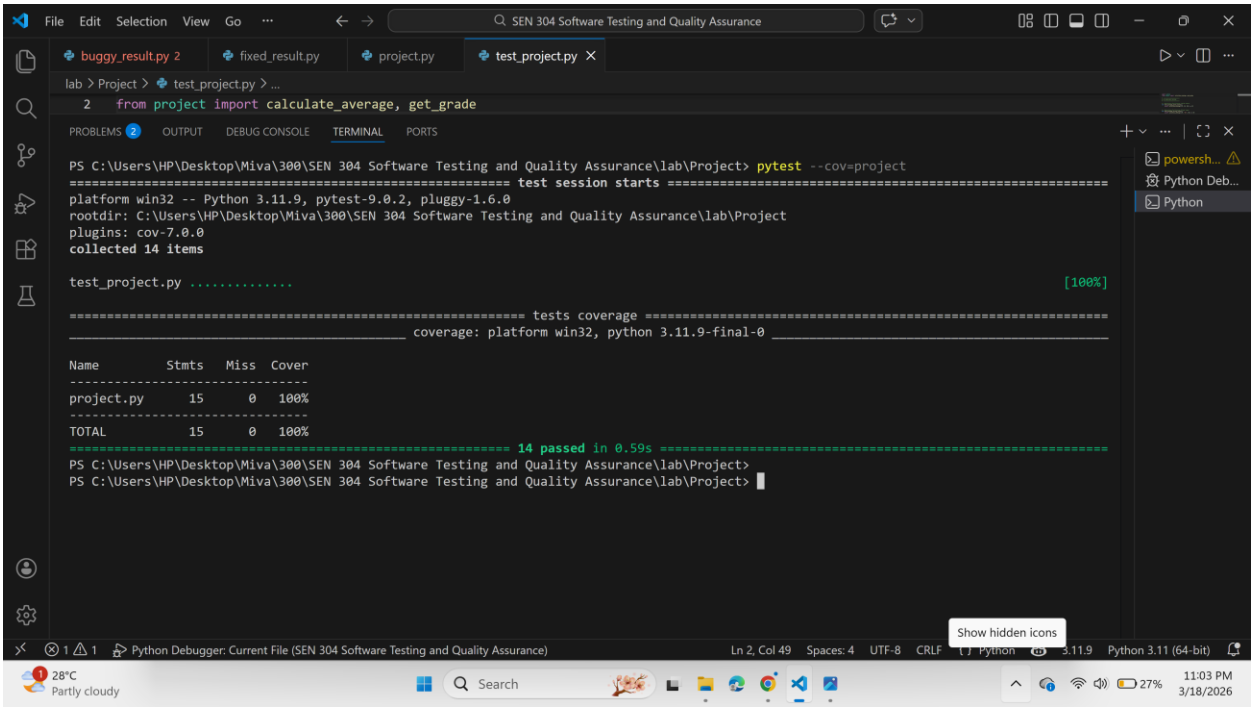
Coverage Result:

```
Name         Stmts   Miss  Cover
--------------------------------
project.py      15      0   100%
--------------------------------
TOTAL           15      0   100%
```

This indicates that:

- All lines of code were executed
- No part of the program was left untested

Screenshot:



5. TEST EXECUTION RESULT

All test cases were executed successfully.

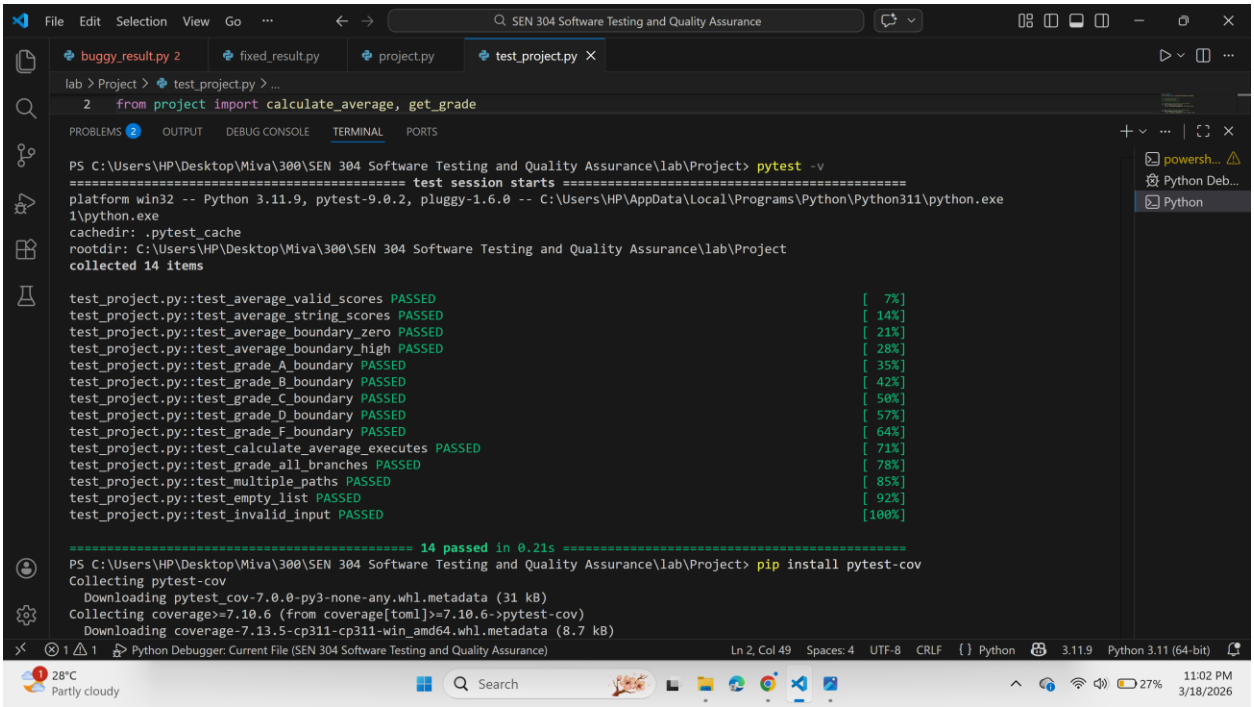
Command Used:

```
pytest -v
```

Result:

- 14 tests collected
- 14 tests passed
- 0 failures

Screenshot:



6. CHALLENGES FACED

During the testing process, the following challenges were encountered:

a. Import Error

- Issue: `attempted relative import with no known parent package`
- Cause: Incorrect use of relative import (`from .project`)
- Solution: Changed to absolute import (`from project import ...`)

b. Coverage Not Detected

- Issue: `No data was collected`
- Cause: Incorrect module name used in coverage command
- Solution: Used correct command:

```
pytest --cov=project
```

c. Handling Invalid Inputs

- Issue: Program crashed when encountering non-numeric values
- Solution: Added test cases to validate error handling using `pytest.raises()`

7. CONCLUSION

The testing process successfully verified the correctness and reliability of the system.

Key achievements include:

- Full implementation of black-box and white-box testing techniques
- Comprehensive test coverage of all functions
- Successful handling of edge cases and errors
- Achieved **100% test coverage**

This demonstrates that the system is robust, well-tested, and meets the required quality standards.

8. SUBMISSION CONTENTS

The following items are included in the submission:

- Source code file (`project.py`)
- Test file (`test_project.py`)
- Screenshots of test execution
- Screenshots of coverage results
- Debugging report (Part A)
- Testing summary report (Part B)